



Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique
Université de Carthage
Institut supérieur des technologies de l'information et de la
communication

Chapitre 4 : Les tableaux

Plan

1. Introduction
2. Tableaux à une dimension
3. Tableaux à deux dimensions

1. Introduction

Introduction du concept :

Exemple : Écrire un algorithme qui calcule la somme de 10 entiers saisis par l'utilisateur.

1ère Solution :

Algorithme Somme

Variables

a, b, c, d, e, f, g, h, i, j, Som : Entier

Début

Lire (a, b, c, d, e, f, g, h, i, j)

Som ← a + b + c + d + e + f + g + h + i + j

Écrire ('La somme = ', Som)

Fin

2ème Solution : Lorsque la résolution d'un problème nécessite l'utilisation d'un nombre assez important de variables de même type, il sera plus facile de déclarer une seule variable et de pouvoir utiliser plusieurs instances de cette variable avec un indice : c'est la déclaration de la variable de type tableau.

Définition du concept :

Un tableau est une structure de données qui permet de manipuler plusieurs valeurs de même type en utilisant un seul nom de variable. Il existe plusieurs types de tableaux. Dans ce cours, nous étudions les tableaux unidimensionnels et les tableaux bidimensionnels (Matrices).

Un tableau possède généralement deux tailles :

- La taille avec laquelle il a été déclaré. C'est la taille maximale puisque elle permet de faire la réservation mémoire nécessaire. Cette taille est soit donnée dans l'énoncé, soit estimé d'après les données de la vie courante ou d'un cas de problème similaire.
- La taille réelle, inférieure ou égale à la taille de déclaration, soit parce que l'utilisateur a choisi de ne pas utiliser toutes les cases réservées, ou encore, parce que certaines valeurs ont été supprimées du tableau et de ce fait, les cases correspondantes ne sont plus accessibles.

2. Tableaux à une dimension

On parle de tableau unidimensionnel lorsque le type des éléments du tableau est un type simple (entier, caractère, réel, booléen, scalaire énuméré...).

Syntaxe de déclaration en Algorithmique:

Nom : tableau [min..max] de <Type>

Avec

Nom : Nom du tableau

Min : indice minimum du tableau

Max : indice maximum du tableau

Type : Type des éléments du tableau.

Exemple : T : tableau [1.. 20] d'entiers

Syntaxe de déclaration en langage C:

```
type Nom_du_tableau [taille maximale] ;
```

Exemples :

```
int A[25]; /* A est un tableau de 25 entiers */  
float B[100]; /* B est un tableau de 100 réels */  
char C[30]; /* C est un tableau de 30 caractères */
```

Remarque : Lors de la déclaration, le tableau est créé, mais les éléments n'ont pas de valeurs. On dit alors que le tableau est vide ou il n'est pas initialisé.

Accès aux éléments :

Dans la mémoire, le tableau est représenté par une suite de cases mémoires.

Nom_tableau [indice] permet d'accéder à l'ième case du tableau, avec i une variable de type scalaire. En algorithmique, la première case est d'indice 1 et la dernière est d'indice N (taille du tableau). En Langage C, les tableaux commencent toujours avec l'indice 0 et finissent avec l'indice N-1.

Initialisation et réservation automatique

Lors de la déclaration d'un tableau, on peut initialiser les composantes du tableau, en indiquant la liste des valeurs respectives entre accolades.

Exemple 1

Il faut évidemment veiller à ce que le nombre de valeurs dans la liste corresponde (soit inférieur ou égal) à la dimension du tableau.

```
int A[5] = {10, 20, 30, 40, 50};  
float B[4] = {-1.05, 3.33, 87e-5, -12.3E4};  
int C[10] = {1, 0, 0, 1, 1, 1, 0, 1, 0, 1};
```

Exemple 2

Si la dimension n'est pas indiquée explicitement lors de l'initialisation, alors l'ordinateur réserve automatiquement le nombre d'octets nécessaires.

```
int A[] = {10, 20, 30, 40, 50, 60, 70};
```

==> Réserve de 7*sizeof(int) octets (dans notre cas: 14 octets)

```
float B[] = {-1.05, 3.33, 87e-5};
```

==> Réserve de 3*sizeof(float) octets (dans notre cas: 12 octets)

```
int C[] = {1, 0, 0, 1, 1, 1, 0, 1, 0, 1};
```

==> Réserve de 10*sizeof(int) octets (dans notre cas: 20 octets)

Traitements standards sur les tableaux

a) Structure de contrôle à propos taille réelle du tableau qui doit être supérieur à zéro et inférieur à la taille maximale

b) Remplissage et affichage des éléments du tableau sont assurés élément par élément. Remplissage est assuré à travers opération `scanf`. L’affichage est assuré à travers l’opération `printf`. Ces traitements nécessitent l’utilisation des structures de contrôle répétitives (généralement les boucles `Pour` puisque le nombre de répétition est connu d’avance).

Remplissage

```
for (cmp=0 ; cmp < taille_tableau ; cmp++)
{
    printf("donner l'élément d'indice %d", cmp) ;
    scanf("format", &nom_tableau[cmp]) ;
}
```

Affichage

```
for (cmp=0 ; cmp < taille_tableau ; cmp++)
{
    printf("format", nom_tableau[cmp]) ;
}
```

Exemple

```
#include <stdio.h>
void main()
{
    int T[100];
    int n;
    do
    {
        printf("donner taille réelle du tableau");
        scanf("%d", &n);
    }while ((n<=0) || (n>100));
    // remplissage
    for (int i=0; i<n;i++)
    {
        printf("donner élément d'indice %d",i);
```

```

        scanf("%d", &T[i]);
    }
    // affichage
    for (int i=0; i<n;i++)
    {
        printf("%d |",T[i]);

    }
}

```

Exercice 1 :

Ecrire un programme permettant de :

- Charger un tableau (tab) par des entiers. Taille maximale du tableau est 100.
- Afficher les éléments du tableau
- Calculer et afficher la somme, produit et moyenne des éléments d'un tableau.
- Déterminer et afficher le maximum et le minimum des éléments d'un tableau.

```

Algorithme caclul_tableau
variables
tab : tableau [1..100] d'entier
n,s, p, min, max: entier
m: réel
début
/*contrôle de saisi sur taille du tableau*/
Répéter
    écrire ("Donner taille du tableau ")
    lire (n)
jusqu'à (n>0 et n<=100)
/*chargement du tableau*/
pour i de 1 à n faire
    écrire ("donner élément d'indice", i)
    lire(tab[i])
fin pour
s←0
p←1
pour i de 1 à n faire
    s←s+ tab[i]
    p←p*tab[i]
fin pour
m←s/n
min←tab[1]
max←tab[1]
pour i de 2 à n faire
    si (tab[i]>max) alors
        max←tab[i]
    sinon
        si (tab[i]<min) alors
            min←tab[i]
        Fin si
    Fin si
fin pour

```

```

écrire ("somme c'est:", s)
écrire ("produit c'est:", p)
écrire ("moyenne c'est:", m)
écrire ("maximum c'est:", max)
écrire ("minimum c'est:", min)
fin

```

Traduction en langage C

```

#include <stdio.h>
void main()
{
int tab[100];
int n, i, min, max;
long S,P; /* somme et produit des éléments - type long à cause de la
grandeur prévisible des résultats. */
float Moy;

/*contrôle de saisi sur taille du tableau*/
do
{
printf("donner le nombre d'élément dans le tableau ");
scanf("%d",&n);
}while( (n<=0)|| (n>100));
/*chargement du tableau*/
for(i=0; i<n; i++)
{
printf("donner l'élément d'indice %d ",i);
scanf("%d", &tab[i]);
}
S=0;
P=1;
for(i=0; i<n; i++)
{
S+=tab[i];
P*=tab[i];
}
Moy=(float)S/n;
max=tab[0];
min=tab[0];
for(i=1; i<n; i++)
{
if (tab[i]>max)
max=tab[i];
else
if (tab[i]<min)
min=tab[i];
}
printf("somme des éléments du tableau est %ld", S);
printf("\n");
printf("produit des éléments du tableau est %ld ", P);
printf("\n");
printf(" Moyenne est %f ", Moy);
printf("\n");
printf(" maximum est %d ", max);
printf("\n");

```

```
printf(" minimum est %d ", min);

}
```

Exercice 2

Un tableau Treel de dimension maximale 40 dont chaque élément est un nombre réel. Écrire un programme en C qui détermine si Treel est trié par valeurs croissante ou non.

```
Algorithme Verif_tri
Variables
Treel: tableau [1..40] de réels
N,i: entier
T:boolean
Début
/*contrôle de saisi sur taille du tableau*/
Répéter
écrire ("donner le nombre d'élément dans le tableau")
lire (N)
jusqu'à ((N>0) et (N<=40))
/*chargement du tableau*/
pour i de 1 à N faire
écrire ("donner élément d'indice", i)
lire (Treel[i])
fin pour
/*vérifier si tableau est trié*/
i←1
T←vrai
tantque ((i<N) et (T=vrai)) faire
si (Treel[i]>Treel[i+1])
    T←faux
sinon
    i←i+1
Fin si
fin tant que
Si (T=faux) alors
    écrire ("tableau non trié")
sinon
    écrire ("tableau trié")
fin si

Fin algorithme
```

Traduction en langage C

```
#include <stdio.h>
#include <stdbool.h>
void main()
{
float Treel[40];
int n, i;
bool V_trie;
/*contrôle de saisi sur taille du tableau*/
do
{
```

```

printf("donner le nombre d'élément dans le tableau ");
scanf("%d",&n);
}while( (n<=0) || (n>40));

/*chargement du tableau*/
for(i=0; i<n; i++)
{
printf("donner l'élément d'indice %d ",i);
scanf("%f", &Treel[i]);
}
V_trie=true;
i=0;
while((i<n-1) &&(V_trie==true))
{
if(Treel[i]>Treel[i+1])
{
V_trie=false;
}
else
i++;
}
if(V_trie==true)
printf("tableau est trié");
else
printf("tableau n'est pas trié");
}

```

Exercice 3

Ecrire un programme permettant de :

- Remplir un tableau T par des entiers strictement positifs. Taille maximale du tableau est 100.
- Afficher les éléments du tableau
- Supprimer les doublons d'un tableau rempli
- Trier les éléments du tableau dans un ordre croissant
- Afficher le tableau trié
- Insérer un élément donné (x) par l'utilisateur dans le tableau trié.
- Afficher le tableau après ajout de l'élément x.

Solution en langage C

```

#include <stdio.h>
void main()
{
int T[100];
int n, i, aux,x;

/*contrôle de saisi sur taille du tableau*/
do
{
printf("donner le nombre d'élément dans le tableau ");

```

```

scanf("%d",&n);
}while( (n<=0) || (n>100));

/*chargement du tableau*/
for(i=0; i<n; i++)
{
printf("donner l'élément d'indice %d ",i);
scanf("%d", &T[i]);
}
// affichage du tableau initial
printf("tableau initial \n");
for (int i=0; i<n;i++)
{
    printf("%d |",T[i]);

}
//suppression des doublons
for(i=0; i<n-1;i++)
    for(int j=i+1; j<=n-1;j++)
        if (T[i]==T[j])
        {
            for (int k=j; k<n; k++)
            {
                T[k]=T[k+1];
            }
            n--;
        }
    printf(" \n");
// affichage du tableau après suppression des doublons
printf("tableau après suppression des doublons \n");
    for (int i=0; i<n;i++)
    {
        printf("%d |",T[i]);

    }
//trier le tableau dans un ordre croissant
for(i=0; i<n-1;i++)
    for(int j=i+1; j<=n-1;j++)
        if (T[i]>T[j])
        {
            // permuter
            aux =T[i];
            T[i]=T[j];
            T[j]=aux;
        }
    printf(" \n");
// affichage du tableau après trie
printf("tableau après trie \n");
    for (int i=0; i<n;i++)
    {
        printf("%d |",T[i]);

    }
// ajout d'un élément dans un tableau
printf("donner élément à insérer \n");
scanf("%d",&x);

```

```
    i=n;
    T[n]=x;
    while (T[i-1]>x)
    {
        T[i]=T[i-1];
        T[i-1]=x;
        i--;
    }

    printf(" \n");
    // affichage du tableau après ajout
    printf("tableau après ajout \n");
    for (int i=0; i<=n;i++)
    {
        printf("%d |",T[i]);
    }
}
```